

ICTS Graduate Course: Numerical Methods (PHY410.5)

Prayush Kumar,^{*} Ritesh Harshe (tutor),[†] Parameswaran Ajith,[‡] and Alorika Kar (tutor)[§]
International Centre for Theoretical Sciences, Tata Institute of Fundamental Research, Bangalore 560089, India.
(Dated: January 29, 2026)

I. INTERPOLATION AND EXTRAPOLATION TECHNIQUES

A. The Runge Phenomenon in Spectroscopy (Polynomial vs. Rational/Spline Interpolation)

In experimental spectroscopy, one often measures the intensity of a spectral line at discrete wavelength intervals. A typical absorption line shape is modeled by a Lorentzian (Cauchy) distribution:

$$L(x) = \frac{1}{1 + 25x^2}$$

Consider the domain $x \in [-1, 1]$, representing a normalized frequency range around the line center.

1. Sample this function at $N = 11$ equidistant points (nodes) in the interval $[-1, 1]$. Perform a global polynomial interpolation of degree $N - 1$ (Lagrange interpolation). Plot the original function and your interpolant on a fine grid.
2. You should observe wild oscillations near the boundaries (the Runge phenomenon). Calculate the maximum error $\epsilon_{max} = \max |L(x) - P_{N-1}(x)|$.
3. Repeat the interpolation using **Chebyshev nodes**, defined as $x_k = \cos\left(\frac{2k-1}{2N}\pi\right)$ for $k = 1, \dots, N$. Plot the result and compare the maximum error.
4. Finally, use a **Cubic Spline** on the original 11 equidistant nodes.
5. *Discussion:* If you were trying to measure the "equivalent width" (area under the curve) of this spectral line, which interpolation method would introduce significant unphysical artifacts, and why?

B. Thermodynamics of Stellar Interiors (Derivatives of Interpolants)

In stellar structure simulations (e.g., White Dwarfs or Neutron Stars), the Equation of State (EOS) is often provided as a pre-computed table of Pressure (P) vs. Density (ρ). However, hydrodynamic stability calculations require the adiabatic sound speed, defined as:

$$c_s^2 = \frac{dP}{d\rho}$$

1. Generate a synthetic "EOS table" using the polytropic relation $P(\rho) = K\rho^{5/3}$ (modeling a non-relativistic degenerate Fermi gas). Create a dataset of 10 points logarithmically spaced between $\rho = 10^4$ and $\rho = 10^7$.
2. **Linear Interpolation:** Interpolate $P(\rho)$ linearly between the points. numerically compute the derivative $dP/d\rho$ (the sound speed squared) from this interpolant on a fine grid. Plot c_s^2 vs ρ .
3. **Cubic Spline:** Interpolate the same data using a Cubic Spline and compute the derivative of the spline. Plot this c_s^2 on the same graph.

^{*}Electronic address: prayush@icts.res.in

[†]Electronic address: ritesh.harshe@icts.res.in

[‡]Electronic address: ajith@icts.res.in

[§]Electronic address: alorika.kar@icts.res.in

4. *Discussion:* What physical violation occurs in the sound speed when using linear interpolation? Why is the Cubic Spline preferred for thermodynamic tables used in hydrodynamics codes?

C. Non-Parametric Inference with Gaussian Processes (Reconstructing the Universe)

In modern data-driven physics (e.g., reconstructing the history of Dark Energy from Supernovae), we often need to interpolate functions where the data is noisy and we do not have a trusted physical theory to define the functional form. Gaussian Process (GP) Regression is the standard tool for this.

1. **Synthetic Data:** Generate a dataset representing a "mystery physical signal" $f(x) = \sin(x) + 0.5x$ on the interval $x \in [0, 10]$. Select 15 random points and add Gaussian noise with $\sigma = 0.3$.
2. **The Kernel:** Perform a GP regression using a **Squared Exponential (RBF) kernel**:

$$k(x, x') = \sigma_f^2 \exp\left(-\frac{(x - x')^2}{2\ell^2}\right)$$

You can use `scikit-learn` or implement the covariance matrix operations directly.

3. **Uncertainty Quantification:** Plot the interpolated mean function and the 95% confidence region (shaded band).
4. **Extrapolation:** Extend the prediction domain to $x \in [0, 15]$.
5. *Discussion:* Compare the behavior of the GP error bars in the extrapolation region ($x > 10$) to what you would get from a high-order polynomial fit. Which method "knows" that it is ignorant?

D. "Infinite" Precision via Richardson Extrapolation (Romberg Integration)

We often cannot perform calculations at the continuum limit ($h \rightarrow 0$), but we can extrapolate to it. Consider the period T of a simple pendulum with length $L = 1$ and $g = 1$, released from a large amplitude $\theta_0 = 0.95\pi$ (very close to the top). The period is given by the elliptic integral:

$$T = 4 \int_0^{\pi/2} \frac{d\phi}{\sqrt{1 - k^2 \sin^2 \phi}}, \quad \text{where } k = \sin(\theta_0/2)$$

1. Write a function to compute this integral using the **Trapezoidal Rule** with a step size h . Calculate the period $T(h)$ for a coarse grid $N = 8$ intervals ($h_1 = \pi/16$) and a finer grid $N = 16$ intervals ($h_2 = \pi/32$).
2. The error in the Trapezoidal rule scales as $O(h^2)$. Use **Richardson Extrapolation** to combine $T(h_1)$ and $T(h_2)$ to produce a new estimate T_{extrap} that cancels the leading order error term.

$$T_{extrap} \approx \frac{4T(h/2) - T(h)}{3}$$

3. Compare the error of $T(h_1)$, $T(h_2)$, and T_{extrap} against the "true" value (computed using `scipy.special.ellipk` or a very high-resolution integration).
4. *Discussion:* Why is this method (often called Romberg Integration when repeated) computationally cheaper than simply running the Trapezoidal rule with $N = 10^6$ points?

II. NUMERICAL CALCULUS

A. Numerical Differentiation

1. Finite differencing, convergence, error estimates

We derived the following finite-differencing approximants for the derivative of a function $f(x)$:

$$\text{Forward differencing : } f'(x) \simeq \frac{f(x+h)-f(x)}{h} + O(h) \quad (2.1)$$

$$\text{Backward differencing : } f'(x) \simeq \frac{f(x)-f(x-h)}{h} + O(h) \quad (2.2)$$

$$\text{Central differencing : } f'(x) \simeq \frac{f(x+h)-f(x-h)}{2h} + O(h^2) \quad (2.3)$$

Problems:

1. Write a Python function to compute derivatives using these three finite differencing methods. Compute the derivative of the function $f(x) = e^x \sin(x)$ over the range $x = [0, 2\pi)$. Plot the numerically computed derivative $f'_{(h)}(x)$ for three different values of h .
2. Plot the error $\Delta f'_h(x) := |f'_h(x) - f'(x)|$ for three different values of h , and estimate the order of convergence n of each finite-difference approximation. Plot $n(x)$, where

$$n(x) = \log_2 \frac{f'_{4h}(x) - f'_{2h}(x)}{f'_{2h}(x) - f'_h(x)} \quad (2.4)$$

3. Reduce the step size h successively. At what value of h does the round-off error dominate the error budget?
4. You would notice that the truncation error is not constant across x . This calls for non-uniform sampling of $f(x)$ in finite differencing. Derive an explicit central differencing formula assuming that $f(x)$ can be sampled at some *non-uniform* samples x_i .

2. Richardson extrapolation

We have seen that, if the order of the error in the numerical estimate of a function $f(x)$ is known, Richardson extrapolation provides a powerful way of improving the accuracy of the estimate. If we have two numerical estimates $f_h(x)$ and $f_{2h}(x)$ each having an error of $O(h^k)$, a better estimate is given by

$$f(x) \simeq \frac{2^k f_h(x) - f_{2h}(x)}{2^k - 1} + O(h^l), \quad (2.5)$$

where l is the next-to-leading-order error term (for e.g., $l = k + 2$ for central differencing, while $l = k + 1$ for forward/backward differencing).

Problems:

1. Gravitational-waves (GWs) have two independent polarization states – called “plus” and “cross” states. GW signals from the coalescence of black-hole binaries, in the simplest case, are circularly polarized:

$$h_+(t) = A(t) \cos \varphi(t), \quad h_\times(t) = A(t) \sin \varphi(t). \quad (2.6)$$

Download the data file [1] containing $h_+(t)$ and $h_\times(t)$. (This is the reduced form of the data produced by a numerical-relativity simulation of black-hole binaries performed by the SXS collaboration and is publicly available at [2]). Compute the phase evolution $\varphi(t)$, the frequency evolution $\omega(t) := d\varphi(t)/dt$ and the rate of change of frequency $\dot{\omega}(t) := d\omega(t)/dt$ using second-order central difference approximation.

2. Estimate the order of convergence of the numerical computation of $\omega(t)$ and $\dot{\omega}(t)$.
3. Perform an extrapolation of $\omega(t)$ and $\dot{\omega}(t)$ to the next order using estimates of two different time-resolutions.
4. Derive an explicit expression for $f'(x)$ with error $O(h^4)$ using Richardson extrapolation. This is the fourth-order finite differencing approximant for the derivative, which we will use later.

B. Numerical integration

1. Newton–Cotes formulas

We derived the following formulas for evaluating the following integral numerically: $I = \int_{x_1}^{x_2} f(x) dx$.

$$\text{Midpoint rule :} \quad I = h f\left(\frac{x_1+x_2}{2}\right) + \mathcal{O}(h^3) \quad (2.7)$$

$$\text{Trapezoidal rule :} \quad I = \frac{h}{2} [f(x_1) + f(x_2)] + \mathcal{O}(h^3) \quad (2.8)$$

$$\text{Simpson's 1/3 rule :} \quad I = \frac{h}{3} [f(x_0) + f(x_2) + 4f(x_1)] + \mathcal{O}(h^5), \quad (2.9)$$

where $h := x_2 - x_1 = x_1 - x_0$ is the width of one slice. Note that the error estimates given above are the local errors. The global error is proportional to the local error / h .

Problems:

1. Show that Trapezoidal rule provides an estimate of the integral with local error $\mathcal{O}(h^3)$. For this you can follow the steps similar to the ones that we used in the class to prove that the Midpoint rule has an error of $\mathcal{O}(h^3)$.
2. Show that Simpson's 1/3 rule provides an estimate of the integral with local error $\mathcal{O}(h^5)$.
3. Compute the following integral numerically using the two explicit methods given above (Trapezoidal and Simpson's)

$$I = \int_{-1}^1 e^{-x^2} dx. \quad (2.10)$$

You may use the following SciPy functions: `scipy.integrate.trapezoid` and `scipy.integrate.simpson`.

4. Repeat the calculation using three different values of h . Plot the error ΔI against h . Show that the numerical estimates converge to the exact value according to the expected order of convergence. First, you can plot the error [which will be $\mathcal{O}(h^n)$] against h as a log-log plot and look at the slope. This will give you an idea of the order of convergence n . Then, you can actually compute n using Eq.(2.4).
5. Using the Trapezoidal and Simpson's methods, compute the energy loss due to GW emission from binary black holes using the numerical-relativity data discussed in Sec. II A 2. The radiated energy can be computed from the GW polarizations as

$$E = \int_{-\infty}^{\infty} \left[\left(\frac{dh_+}{dt} \right)^2 + \left(\frac{dh_{\times}}{dt} \right)^2 \right] dt. \quad (2.11)$$

6. Repeat the calculation with three different values of h (sampling rate dt of the data). Compute the order of convergence [see, e.g., Eq.(2.4)].

2. Romberg method, Adaptive quadrature, Gaussian quadrature

Problems:

1. Repeat problem 3 of Sec. II B 1 using the adaptive Gaussian quadrature. You may use the function `scipy.integrate.quad`.
2. Repeat problem 5 of Sec. II B 1 using the Romberg's method. You may use the function `scipy.integrate.romb`.

III. ORDINARY DIFFERENTIAL EQUATIONS: INITIAL VALUE PROBLEMS

A. Gravitational waves from inspiralling compact binaries

The time evolution of the orbital phase $\varphi(t)$ of a binary of black holes evolving under the gravitational radiation reaction can be computed, in the post-Newtonian approximation, by solving the following coupled ODEs:

$$\frac{dv}{dt} = -\frac{\mathcal{F}(v)}{dE(v)/dv}, \quad \frac{d\varphi}{dt} = \frac{v^3}{m}, \quad (3.1)$$

where $E(v)$ is the binding energy of the orbit, $\mathcal{F}(v)$ is the energy flux of radiated gravitational waves, $m := m_1 + m_2$ is the total mass of the binary, $v = (m\omega)^{1/3}$, ω being the orbital frequency. (Here we use geometric units, in which $G = c = 1$. This means that in all the expressions m has to be replaced by Gm/c^3 .)

The binding energy and gravitational-wave flux are given as post-Newtonian expansions in terms of the small parameter v

$$E(v) = -\frac{1}{2}\mu v^2 [1 + \mathcal{O}(v^2)], \quad \mathcal{F}(v) = \frac{32}{5} \left(\frac{\mu}{m}\right)^2 v^{10} [1 + \mathcal{O}(v^2)], \quad (3.2)$$

where $\mu := m_1 m_2 / m$ is the reduced mass of the system.

1. Compute v as a function of t by solving the first equation in Eq. (3.1) using Scipy's [integrate.RK45](#) routine. This uses the adaptive Runge-Kutta method by Dormand & Prince that is very similar to the Runge-Kutta-Fehlberg method that we learned in the class. Assume the following parameters: $m_1 = m_2 = 5M_\odot$, $v_0 = 0.3$, $\varphi_0 = 0$. Plot $v(t)$.
2. Solve the coupled system in Eq. (3.1) to compute v and φ . Compute and plot the two gravitational-wave polarizations:

$$h_+(t) = 4 \frac{\mu}{m} v^2 \cos \varphi(t), \quad h_\times(t) = 4 \frac{\mu}{m} v^2 \sin \varphi(t). \quad (3.3)$$

B. Structure of a relativistic, spherically symmetric star

The interior structure of a relativistic, spherically symmetric star is described by a metric that has the line element

$$ds^2 = -e^{2\Phi(r)} c^2 dt^2 + \left(1 - \frac{2Gm(r)}{rc^2}\right)^{-1} dr^2 + r^2 d\Omega^2, \quad (3.4)$$

where $m(r)$ is called the *mass function* (which encapsulates the gravitational mass inside the radius r), $e^{2\Phi(r)}$ is the *lapse function* (which relates the proper time with the coordinate time). Outside the “surface” of the star (in vacuum), the spacetime is described by the Schwarzschild metric; the lapse function becomes

$$\Phi(r) = \frac{1}{2} \ln \left(1 - \frac{2Gm_\star}{rc^2}\right) \quad (3.5)$$

where $m_\star$ is the total (gravitational) mass of the star. The structure can be computed by solving the following set of ordinary differential equations, derived by Tolman, Oppenheimer and Volkoff (TOV).

$$\frac{dm(r)}{dr} = 4\pi r^2 \rho(r), \quad (3.6)$$

$$\frac{dP(r)}{dr} = -\frac{G(\rho + P(r)/c^2)}{r^2} \left[m(r) + \frac{4\pi r^3 P(r)}{c^2} \right] \left[1 - \frac{2Gm(r)}{c^2 r} \right]^{-1}, \quad (3.7)$$

$$\frac{d\Phi(r)}{dr} = \frac{Gm(r) + 4\pi Gr^3 P(r)/c^2}{c^2 r [r - 2Gm(r)/c^2]}. \quad (3.8)$$

The TOV equations have to be supplemented by an *equation of state* $P = P(\rho)$, that relates the pressure P to the energy density ρ . We assume the equation of state to be of polytropic form

$$P(r) = K \rho(r)^\gamma. \quad (3.9)$$

We also need to specify initial conditions for the variables m, P, Φ . The following conditions can be used

$$m(r=0) = 0, \quad P(r=0) = P_c = P(\rho_c), \quad \Phi(r_\star) = \frac{1}{2} \ln \left(1 - \frac{2Gm_\star}{r_\star c^2}\right). \quad (3.10)$$

For the problems in this section, you may use Scipy's high-level interface to various ODE solvers [solve_ivp](#).

Problems:

1. Compute the structure, i.e., $m(r)$ and $P(r)$, of a neutron star with central density $\rho_c = 5 \times 10^{17} \text{ kg/m}^3$ by solving Eqs. (3.6) and (3.7). Assume a polytropic equation of state with $\gamma = 5/3$ and $K = 5380.3$ (SI units). What is the mass $m_\star$ and radius $r_\star$ of the neutron star? (Useful tip: You will need to start the integration at $r = \Delta r$, where Δr is a small number. You can assume $m(r = \Delta r) := 4/3 \pi \rho_c (\Delta r)^3$).
2. Compute the lapse function $e^{2\Phi(r)}$ by solving Eq.(3.8) starting from $r = r_\star$ to $r = 0$. On top of that, plot the lapse function for a Schwarzschild black hole (see Eq.3.5) from $r = r_s$ to $r = 2r_\star$, where $r_s \equiv 2Gm_\star/c^2$ is the Schwarzschild radius of the star. This exterior solution should match the interior solution at $r = r_\star$.

C. Non-linear ordinary differential equations showing chaotic behavior: Lorenz equations

The Lorenz equations were originally developed as a simplified mathematical model for atmospheric convection by Edward Lorenz. This was the first set of equations where deterministic chaos was observed. These coupled ordinary differential equations are

$$\begin{aligned} \frac{dx(t)}{dt} &= \sigma[y(t) - x(t)], \\ \frac{dy(t)}{dt} &= x(t)[\rho - z(t)] - y(t), \\ \frac{dz(t)}{dt} &= x(t)y(t) - \beta z(t), \end{aligned} \tag{3.11}$$

where ρ, σ and β are parameters of the system.

1. Problems:

1. Solve the Lorenz system for $\rho = 28, \sigma = 10$ and $\beta = 8/3$ with the following initial conditions $x(t = 0) = y(t = 0) = z(t = 0) = 1$. Plot $x(t), y(t)$ and $z(t)$ for $t = 0 \dots 100$. Is the solution deterministic or stochastic?
2. Repeat the calculation with same parameters except for a tiny change in the initial condition for x : i.e., $x(t = 0) = 1 + 10^{-9}$. Plot $x(t = 0) = y(t = 0) = z(t = 0) = 1$ on top of the earlier estimate. Explain the result.
3. Make a 3D plot of x, y, z . You should see the famous butterfly shaped structure now!
4. Optional exercise: Make an animation of the above ¹.

IV. ROOT FINDING

The Kepler's equation is a transcendental equation describing planetary motion:

$$M - E + e \sin E = 0. \tag{4.1}$$

This gives the relation between the mean anomaly M (a parametrisation of time) and the eccentric anomaly E (parametrisation of the polar angle of the planet, given the orbital eccentricity e). The mean anomaly can be expressed in terms of the time coordinate t as

$$M = \frac{2\pi}{T}(t - \tau), \tag{4.2}$$

where T is the period of the orbit and τ is the time when the planet reaches the periapsis. Combining this with the Kepler's equation, we can find E as a function of t . The coordinates of the planet are then given by

$$x = a(\cos E - e), \quad y = b \sin E, \tag{4.3}$$

where a and b are the semi-major and semi-minor axes, respectively.

¹ You can either use the matplotlib [animation](#) package or convert a number of PNG files to a gif animation using [ImageMagick](#).

1. Problems:

1. Solve the Kepler's equation using the Newton-Raphson's method. Plot E as a function of M for three different values of $e = 0, 0.1, 0.5$. Take M to be in the range $[0, 2\pi)$. You can use scipy's `optimize.root_scalar` function to find the roots.
2. Plot the orbits of solar system planets. The necessary data can be downloaded from the [NASA Planetary Fact Sheet](#).
3. Optional exercise: Make an animation of the above.

V. ORDINARY DIFFERENTIAL EQUATIONS: TWO-POINT BOUNDARY VALUE PROBLEMS

Solve the TOV equations described in Sec. III B as a two-point boundary value problem using the Shooting method. Use the Newton-Raphson method for root finding. The boundary conditions are given in Eq. (3.10).

VI. CURVE FITTING

A. Linear regression

We have seen in the class that, given a theoretical model $Y(x, \mathbf{a}) = \sum_{j=1}^M a_j Y_j(x)$ and a data set $\{x_i, y_i\}$ with associated errors σ_i , we can estimate the parameters $\mathbf{a}$ of the model by minimising the cost function (chi-square). This amounts to solving the following Normal equations of the least square problem

$$(\mathbf{A}^T \mathbf{A}) \mathbf{a} = \mathbf{A}^T \mathbf{b}, \quad (6.1)$$

where the elements of the matrix $\mathbf{A}$ and vector $\mathbf{b}$ are given by

$$A_{ij} = \frac{Y_j(x_i)}{\sigma_i}, \quad b_i = \frac{y_i}{\sigma_i}. \quad (6.2)$$

1. Problems

1. Show that the covariance matrix of the estimated parameters $\mathbf{a}$ are given by $\mathbf{C} = (\mathbf{A}^T \mathbf{A})^{-1}$
2. The Hubble's law provides a relation between the luminosity distance d_L and cosmological redshift z that is valid in the local universe ($z \lesssim 0.1$).

$$d_L = \frac{c}{H_0} z, \quad (6.3)$$

where c is the speed of light and H_0 is the Hubble constant. Here [3] you are given a dataset from the Supernova Cosmology project [4]. This contains the redshift z , the distance modulus μ , and the error on the distance modulus $\delta\mu$ measured from several Type 1a supernovae. The distance modulus is related to the luminosity distance (in parsecs) by $\mu = 5(\log_{10} d_L - 1)$. Use the linear regression method to fit the distance and redshift data to estimate the Hubble constant and the associated error. Note that the data contain redshifts above the range where the simple Hubble law is valid. Please select the samples with $z \leq 0.1$.

B. Non-linear least square fitting

In the flat Λ CDM model of cosmology, the general relation between the luminosity distance d_L and cosmological redshift z is given by

$$d_L = (1+z) \frac{c}{H_0} \int_0^z \frac{dz'}{\sqrt{\Omega_M(1+z')^3 + \Omega_\Lambda}}, \quad (6.4)$$

where Ω_M is the energy density of matter and Ω_Λ is the energy density of the cosmological constant. Since we are assuming spatially flat universe: $\Omega_M + \Omega_\Lambda = 1$. Thus the only free parameters of the model are H_0 and Ω_M .

1. Problems

1. Use the supernova data from the entire redshift to estimate H_0 and Ω_M and the corresponding covariance. This is a non-linear least square problem. You can use SciPy's [curve_fit](#) function.

VII. STATISTICAL INFERENCE

Given some data d and a model $\mathcal{M}$, Bayesian parameter estimation involves computing the posterior distribution of the parameters θ describing the model. Using Bayes theorem, we can write

$$p(\theta|d, \mathcal{M}) = \frac{p(\theta|\mathcal{M}) p(d|\theta, \mathcal{M})}{p(d|\mathcal{M})}, \quad (7.1)$$

where $p(\theta|\mathcal{M})$ is the prior distribution of the parameters θ , $p(d|\theta, \mathcal{M})$ is the likelihood of data given the parameters θ and the model $\mathcal{M}$, while

$$p(d|\mathcal{M}) = \int p(\theta|\mathcal{M}) p(d|\theta, \mathcal{M}) d\theta \quad (7.2)$$

is the evidence of the model $\mathcal{M}$. Bayesian model selection involves comparing the evidence of different models, say $\mathcal{M}_1$ and $\mathcal{M}_2$, by means of the likelihood ratio (Bayes factor) between the two models.

$$\mathcal{B}_2^1 = \frac{p(d|\mathcal{M}_1)}{p(d|\mathcal{M}_2)}. \quad (7.3)$$

1. Problems

Here we solve the curve fitting problem presented in Sec. VI using of Bayesian statistical inference.

1. Compute the posterior distribution $p(H_0|d, \mathcal{M}_1)$ of the Hubble constant H_0 using the Supernova Cosmology project data $z \leq 0.1$. Assume the Hubble's law [Eq. (6.3)] as the model $\mathcal{M}_1$. Assume uniform prior for H_0 in the interval (10, 100) km/s/Mpc. You can either use a Gaussian likelihood

$$p(d|H_0, \mathcal{M}_1) = \exp\left(-\sum_i \left|\frac{d_i - d_L(z_i, \mathcal{M}_1)}{2\sigma_i^2}\right|^2\right), \quad (7.4)$$

where d_i and z_i are the samples of the luminosity distance and redshift from the data and $d_L(z_i, \mathcal{M}_1)$ is the relation between luminosity distance and redshift predicted by model $\mathcal{M}_1$ evaluated at redshift z_i . σ_i is the standard deviation of the measurement in d_L which can be calculated using the error in the distance modulus μ (given in the data file [3]).

2. Repeat the analysis using the full data set. What are the differences that you see in the posterior?
3. Compute the posterior distribution $p(H_0, \Omega_M|d, \mathcal{M}_2)$ of the Hubble constant H_0 and matter density Ω_M using the Λ CDM model $\mathcal{M}_2$ [Eq. (6.4)]. You can compute the posterior on a 2-dimensional grid. Assume uniform priors for H_0 in the interval (10, 100) km/s/Mpc and for Ω_M in the interval (0, 1).
4. Compute the likelihood ratio (Bayes factor) between the Hubble's law and Λ CDM model by computing the evidences [Eq. (7.2)] of the two models using a numerical integration method that we learned in Sec. ??.

VIII. MONTE-CARLO METHODS

1. Problems:

1. The area of a circle of radius r is $A_c = \pi r^2$. The area of the smallest square that encloses this circle is $A_s = 4r^2$. Measure the areas of the circle and square using a Monte-Carlo simulation and estimate the value of $\pi = 4A_c/A_s$.
2. Repeat the problems 3 and 4 from Sec. VII by stochastically sampling the posterior using Markov-Chain Monte Carlo and then computing the evidence using Monte-Carlo integration. You can either code up the Metropolis-Hastings algorithm or use an existing sampler such as [dynesty](#).

IX. FOURIER AND SPECTRAL METHODS

A. Fast Fourier transform (FFT)

1. Problems:

1. Prove the convolution theorem.
2. Prove the correlation theorem.
3. Compute the Fourier transform of the function $h(t) = \sin 2\pi f_0 t$ where $f_0 = 10$ Hz, $t = [0, 30]$ seconds using `numpy.fft`. Use different window functions and compare the Fourier transforms obtained using different window functions.
4. Hercules X-1 is a high-mass X-ray binary (HMXB) system having a magnetized spinning neutron star which happens to be an X-ray pulsar. Here [5] you are given the data of an RXTE [6] observation after cleaning and pre-processing. The file contains two columns, (i) time (in seconds) and (ii) count-rate of X-ray photons (i.e., number of photons detected per unit time). Plot the count-rate as function of time. This is called lightcurve in X-ray astronomy. Compute the FFT of the given time-series and plot its absolute value against the frequency. Are you able to find any periodic signal(s)? The lowest frequency signal corresponds to the spin frequency of the neutron star. What is the spin period of this pulsar? Can you identify other periodic signals and how they are related to the lowest frequency signal?²

B. Power spectrum estimation using FFT

1. Problems:

1. A sample data set from the LIGO gravitational-wave observatory can be downloaded from here [7]. This contains 256 seconds of LIGO data from 2005, sampled at a rate of 4096 Hz. Compute the power spectral density of the data using Welch's modified periodogram method.

-
- [1] URL http://home.icts.res.in/~ajith/Downloads/nr_data.gz.
 [2] URL <http://www.black-holes.org/waveforms/>.
 [3] URL https://home.icts.res.in/~ajith/Downloads/SCPUnion2.1_mu_vs_z.txt.
 [4] URL <https://supernova.lbl.gov/>.
 [5] URL http://home.icts.res.in/~ajith/Downloads/extracted_lightcurve_HerX-1.dat.gz.
 [6] RXTE is an X-ray timing satellite, URL <https://heasarc.gsfc.nasa.gov/docs/xte/rxte.html>.
 [7] Download the file L1-STRAIN_4096Hz-815045078-256.txt.gz from, URL <http://www.ligo.org/science/GRB051103/index.php>.

² This problem and data are graciously provided by Dr. Arunava Mukherjee (SINP).